

TaskBoard

Day 6 ? Lab 6.3 ? Protected Routes

Source code and project configuration

32 files included ? Generated from the final Lab 6.3 checkpoint

eslint.config.js

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import tseslint from 'typescript-eslint'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.ts', '**/*.tsx'],
    extends: [
      js.configs.recommended,
      tseslint.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      globals: globals.browser,
    },
  },
])
```

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>TaskBoard</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

package.json

```
{
  "name": "taskboard-ts",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.8",
    "react-dom": "^19.2.8",
    "react-router": "^8.4.0"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/node": "^24.13.3",
    "@types/react": "^19.2.18",
    "@types/react-dom": "^19.2.7",
    "@vitejs/plugin-react": "^6.1.1",
    "eslint": "^10.10.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.6",
    "globals": "^17.12.0",
    "typescript": "~6.0.2",
    "typescript-eslint": "^8.69.0",
    "vite": "^8.3.0"
  }
}
```

README.md

TaskBoard: Lab 6.3 protected-routes checkpoint

This checkpoint adds a local mock login to the routed TaskBoard. A logged-out visitor who opens Settings is redirected to Login; after login they are returned to Settings. The mock user is stored in localStorage and is not real authentication.

Run

```
```bash
npm install
npm run dev
```
```

Use `npm run lint` to run ESLint and `npm run build` to type-check and build. The first run loads starter tasks from JSONPlaceholder, so it needs an internet connection.

src/App.css

```
/* TODO0 (Lab 7.1 step 3): replace the hard-coded colours below with your CSS variables so dark mode reaches every component. */
#root {
  max-width: 960px;
  margin: 0 auto;
  padding: 24px 16px;
}

/* Lab 3.1: Button */
.btn {
  font: inherit;
  border-radius: 6px;
  border: 1px solid transparent;
  cursor: pointer;
  margin-right: 8px;
}

.btn-sm {
  padding: 4px 8px;
  font-size: 0.875rem;
}

.btn-md {
  padding: 8px 16px;
}

.btn-lg {
  padding: 12px 20px;
  font-size: 1.125rem;
}

.btn-primary {
  background: #0f766e;
  color: #ffffff;
}

.btn-secondary {
  background: #ffffff;
  color: #0f766e;
  border-color: #0f766e;
}

/* Lab 3.1: Card */
.card {
  background: #ffffff;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  padding: 16px;
  margin-bottom: 16px;
}

.card h2 {
  margin-top: 0;
}

.card-highlight {
  border-color: #0f766e;
}

/* Lab 3.3: TaskBoard */
.header {
  margin-bottom: 16px;
}
```

```

.header {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
}

.header p {
  margin: 0;
}

.brand {
  font-size: 1.5rem;
  font-weight: 700;
}

.board {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}

.column {
  background: #eef2f7;
  border-radius: 8px;
  padding: 12px;
}

.column h2 {
  margin-top: 0;
  font-size: 1.125rem;
}

.column ul {
  list-style: none;
  margin: 0;
  padding: 0;
}

.column .card {
  margin-bottom: 12px;
}

.column .card h3 {
  margin: 0 0 4px;
  font-size: 1rem;
}

.column .card p {
  margin: 0 0 4px;
}

/* Lab 4.3: interactive TaskBoard */
.add-task-form {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 8px;
  margin-bottom: 16px;
}

.add-task-form input {
  font: inherit;
  padding: 4px 8px;
}

```

```

.add-task-form input[type="number"] {
  width: 64px;
}

.add-task-form p[role="alert"] {
  flex-basis: 100%;
  margin: 0;
  color: #991b1b;
}

.add-task-form button,
.column .card button,
.column .card select {
  font: inherit;
}

.column .card button,
.column .card select {
  margin: 4px 6px 0 0;
}

.column .card > span {
  display: block;
  font-size: 0.875rem;
  font-weight: 600;
  color: #0f766e;
}

/* Lab 6.2: layout, navigation and search */
nav {
  display: flex;
  flex-wrap: wrap;
  gap: 16px;
  padding: 8px 0;
  margin-bottom: 16px;
  border-bottom: 1px solid #e5e7eb;
}

nav a {
  color: #0f766e;
}

.search {
  display: flex;
  align-items: center;
  gap: 8px;
}

.search input {
  font: inherit;
  padding: 4px 8px;
}

.hint {
  font-size: 0.875rem;
}

/* Lab 6.3: mock login */
nav .user {
  margin-left: auto;
}

nav .user button {
  font: inherit;
}

```

```
.login-form {
  display: grid;
  gap: 8px;
  max-width: 320px;
}

.login-form input,
.login-form button {
  font: inherit;
  padding: 4px 8px;
}

/* Lab 5.3: persistence and seed data */
.reset-button {
  font: inherit;
  margin-bottom: 16px;
}
```

src/components/AddTaskForm.tsx

```
// TODO (Lab 7.2 step 6): call the store's addTask (generate the id here, not in the reducer).
// TODO (Lab 7.3 steps 6-7): call useAddTask().mutate instead, and disable the button while it is pending.
import { useEffect, useRef, useState } from "react";
import type { ChangeEvent, SubmitEvent } from "react";

export type NewTaskFields = {
  title: string;
  assignee: string;
  points: number;
};

type AddTaskFormProps = {
  onAdd: (task: NewTaskFields) => void;
};

type FormState = {
  title: string;
  assignee: string;
  points: string;
};

const emptyForm: FormState = { title: "", assignee: "", points: "1" };

function AddTaskForm({ onAdd }: AddTaskFormProps) {
  const [form, setForm] = useState<FormState>(emptyForm);
  const [error, setError] = useState("");
  const inputRef = useRef<HTMLInputElement>(null);

  useEffect(() => {
    inputRef.current?.focus();
  }, []);

  function handleChange(e: ChangeEvent<HTMLInputElement>) {
    const { name, value } = e.target;
    setForm((prev) => ({ ...prev, [name]: value }));
  }

  function handleSubmit(e: SubmitEvent<HTMLFormElement>) {
    e.preventDefault();
    const title = form.title.trim();
    if (title.length < 3) {
      setError("Title needs 3+ characters.");
      return;
    }
    setError("");
    onAdd({
      title,
      assignee: form.assignee.trim(),
      points: Number(form.points),
    });
    setForm(emptyForm);
  }

  return (
    <form className="add-task-form" onSubmit={handleSubmit}>
      <label htmlFor="title">Title</label>
      <input id="title" name="title" ref={inputRef} value={form.title}
        aria-invalid={Boolean(error)} onChange={handleChange} />

      <label htmlFor="assignee">Assignee</label>
      <input id="assignee" name="assignee" value={form.assignee}
        onChange={handleChange} />
    </form>
  );
}
```

```

    <label htmlFor="points">Points</label>
    <input id="points" name="points" type="number" min="1"
      value={form.points} onChange={handleChange} />

    {error && <p role="alert">{error}</p>}
    <button type="submit">Add task</button>
  </form>
);
}

export default AddTaskForm;

```

src/components/Board.tsx

```

// TODO (Lab 7.2 step 5): remove the tasks and handler props.
// TODO (Lab 7.3 step 7): show loading and error states from useQuery.
import Column from "../Column";
import type { Status, Task } from "../types";

type BoardProps = {
  tasks: Task[];
  onStatusChange: (id: string, status: Status) => void;
  onRename: (id: string, title: string) => void;
  onDelete: (id: string) => void;
};

const columns: [Status, string][] = [
  ["todo", "To do"],
  ["doing", "In progress"],
  ["done", "Done"]
];

function Board({ tasks, onStatusChange, onRename, onDelete }: BoardProps) {
  return (
    <div className="board">
      {columns.map(([status, heading]) => (
        <Column
          key={status}
          heading={heading}
          tasks={tasks.filter((t) => t.status === status)}
          onStatusChange={onStatusChange}
          onRename={onRename}
          onDelete={onDelete}
        />
      ))}
    </div>
  );
}

export default Board;

```

src/components/Column.tsx

```
// TODO (Lab 7.2 step 6): select the tasks with useTaskStore((s) => s.tasks) and filter them here, outside the selector.
// TODO (Lab 7.3 step 5): read the tasks with useQuery({ queryKey: ["tasks"], queryFn: fetchTasks }).
import TaskCard from "../TaskCard";
import type { Status, Task } from "../types";

type ColumnProps = {
  heading: string;
  tasks: Task[];
  onStatusChange: (id: string, status: Status) => void;
  onRename: (id: string, title: string) => void;
  onDelete: (id: string) => void;
};

function Column({ heading, tasks, onStatusChange, onRename, onDelete }: ColumnProps) {
  return (
    <section className="column">
      <h2>{heading} ({tasks.length})</h2>
      {tasks.length === 0 ? (
        <p>Nothing here yet</p>
      ) : (
        <ul>
          {tasks.map((task) => (
            <li key={task.id}>
              <TaskCard
                task={task}
                onStatusChange={onStatusChange}
                onRename={onRename}
                onDelete={onDelete}
              />
            </li>
          ))}
        </ul>
      )}
    </section>
  );
}

export default Column;
```

src/components/Header.tsx

```
// TODO (Lab 7.2 step 6): read the tasks from the store instead of a prop.
// TODO (Lab 7.3 step 5): read them with useQuery.
import { useEffect } from "react";
import type { Task } from "../types";

type HeaderProps = {
  tasks: Task[];
};

function Header({ tasks }: HeaderProps) {
  const doneCount = tasks.filter((t) => t.status === "done").length;
  const openCount = tasks.filter((t) => t.status !== "done").length;

  useEffect(() => {
    document.title = `TaskBoard (${openCount} open)`;
  }, [openCount]);

  return (
    <header className="header">
      <p className="brand">TaskBoard</p>
      <p>{doneCount} of {tasks.length} done</p>
    </header>
  );
}

export default Header;
```

src/components/ProjectBoard.tsx

```
// TODO (Lab 7.2 steps 5-6): no more handlers passed as props. AddTaskForm and TaskCard call store actions
// themselves, so this component is no longer needed.
import { useOutletContext } from "react-router";
import AddTaskForm from "../AddTaskForm";
import type { NewTaskFields } from "../AddTaskForm";
import Board from "../Board";
import type { BoardContext } from "../pages/Layout";
import type { Status, Task } from "../types";

type ProjectBoardProps = {
  tasks: Task[];
  projectId: string;
};

// The add form and board, wired to the tasks state that Layout shares
// through Outlet context. Dashboard and Project both render it.
export default function ProjectBoard({ tasks, projectId }: ProjectBoardProps) {
  const { setTasks } = useOutletContext<BoardContext>();

  function handleAdd(newTask: NewTaskFields) {
    const id = crypto.randomUUID();
    setTasks((prev) => [...(prev ?? []), { ...newTask, id, status: "todo", projectId }]);
  }

  function handleStatusChange(id: string, status: Status) {
    setTasks((prev) =>
      (prev ?? []).map((t) => (t.id === id ? { ...t, status } : t))
    );
  }

  function handleRename(id: string, title: string) {
    setTasks((prev) =>
      (prev ?? []).map((t) => (t.id === id ? { ...t, title } : t))
    );
  }

  function handleDelete(id: string) {
    setTasks((prev) => (prev ?? []).filter((t) => t.id !== id));
  }

  return (
    <>
      <AddTaskForm onAdd={handleAdd} />
      <Board
        tasks={tasks}
        onStatusChange={handleStatusChange}
        onRename={handleRename}
        onDelete={handleDelete}
      />
    </>
  );
}
```

src/components/RequireAuth.tsx

```
import type { ReactNode } from "react";
import { Navigate, useLocation } from "react-router";
import { useAuth } from "../hooks/useAuth";

export default function RequireAuth(
  { children }: { children: ReactNode }
) {
  const { user } = useAuth();
  const location = useLocation();

  if (!user) {
    return <Navigate to="/login" replace
      state={{ from: location.pathname }} />;
  }
  return children;
}
```

src/components/TaskCard.tsx

```
// TODO (Lab 7.2 step 6): select moveTask, renameTask and deleteTask from the store instead of props.
// TODO (Lab 7.3 steps 6-7): use the useMoveTask and useDeleteTask mutations, and disable controls while they are pending.
import { useState } from "react";
import type { ChangeEvent } from "react";
import type { Status, Task } from "../types";

type TaskCardProps = {
  task: Task;
  onStatusChange: (id: string, status: Status) => void;
  onRename: (id: string, title: string) => void;
  onDelete: (id: string) => void;
};

function TaskCard({ task, onStatusChange, onRename, onDelete }: TaskCardProps) {
  const [isEditing, setIsEditing] = useState(false);
  const [draft, setDraft] = useState(task.title);

  function handleSave() {
    const title = draft.trim();
    if (title === "") return;
    onRename(task.id, title);
    setIsEditing(false);
  }

  function handleDeleteClick() {
    if (window.confirm(`Delete "${task.title}"?`)) {
      onDelete(task.id);
    }
  }

  function handleStatusChange(e: ChangeEvent<HTMLSelectElement>) {
    onStatusChange(task.id, e.target.value as Status);
  }

  return (
    <article className="card">
      {isEditing ? (
        <>
          <input aria-label="Task title" value={draft}
            onChange={(e) => setDraft(e.target.value)} />
          <button onClick={handleSave}>Save</button>
        </>
      ) : (
        <>
          <h3>{task.title}</h3>
          <button onClick={() => setIsEditing(true)}>Edit</button>
        </>
      )}

      {task.assignee && <p>Assigned to {task.assignee}</p>}
      {task.points > 0 && <span>{task.points} pts</span>}

      <select aria-label="Status" value={task.status} onChange={handleStatusChange}>
        <option value="todo">To do</option>
        <option value="doing">In progress</option>
        <option value="done">Done</option>
      </select>

      <button onClick={handleDeleteClick}>Delete</button>
    </article>
  );
}
```

```
export default TaskCard;
```

src/components/ui/Button.tsx

```
import type { ReactNode } from "react";

type ButtonProps = {
  variant?: "primary" | "secondary";
  size?: "sm" | "md" | "lg";
  children: ReactNode;
};

function Button({ variant = "primary", size = "md", children }: ButtonProps) {
  return <button className={`btn btn-${variant} btn-${size}`}>{children}</button>;
}

export default Button;
```

src/components/ui/Card.tsx

```
import type { ReactNode } from "react";

type CardProps = {
  title?: string;
  variant?: "default" | "highlight";
  children: ReactNode;
};

function Card({ title, variant = "default", children }: CardProps) {
  return (
    <section className={`card card-${variant}`}>
      {title && <h2>{title}</h2>}
      <div className="card-body">{children}</div>
    </section>
  );
}

export default Card;
```

src/data/projects.ts

```
export type Project = { id: string; name: string };

export const projects: Project[] = [
  { id: "website", name: "Website" },
  { id: "mobile", name: "Mobile app" },
];
```

src/data/seed.ts

```
// TODO (Lab 7.2): seeding lived in Layout. Once tasks move to the store and then to json-server (Lab 7.3),
// the starter tasks live in db.json instead.
import type { Task } from "../types";

export const SEED_URL = "https://jsonplaceholder.typicode.com/todos?_limit=5";

export type Todo = { id: number; title: string; completed: boolean };

export function toTasks(todos: Todo[]): Task[] {
  return todos.map((t) => ({
    id: String(t.id),
    title: t.title,
    status: t.completed ? "done" : "todo",
    points: 1,
    projectId: t.id % 2 === 1 ? "website" : "mobile",
  }));
}
```

src/hooks/useAuth.ts

```
// TODO (Lab 7.1 steps 5-6): move user, login and logout into an AuthProvider in src/context,
// and change useAuth to read AuthContext with useContext (throw a clear error outside the provider).
import { useLocalStorage } from "../useLocalStorage";

export type User = { name: string };

export function useAuth() {
  const [user, setUser] = useLocalStorage<User | null>("user", null);

  function login(name: string) {
    setUser({ name });
  }

  function logout() {
    setUser(null);
  }

  return { user, login, logout };
}
```

src/hooks/useFetch.ts

```
import { useEffect, useState } from "react";

type FetchState<T> = {
  data: T | null;
  error: string | null;
  loading: boolean;
};

export function useFetch<T>(url: string): FetchState<T> {
  const [data, setData] = useState<T | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const ctrl = new AbortController();
    fetch(url, { signal: ctrl.signal })
      .then((res) => {
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        return res.json();
      })
      .then((d: T) => { setData(d); setError(null); })
      .catch((e: unknown) => {
        if (!ctrl.signal.aborted) setError(e instanceof Error ? e.message : String(e));
      })
      .finally(() => {
        if (!ctrl.signal.aborted) setLoading(false);
      });
    return () => { ctrl.abort(); setLoading(true); };
  }, [url]);

  return { data, error, loading };
}
```

src/hooks/useLocalStorage.ts

```
// TODO (Lab 7.1 steps 4-5): ThemeProvider and AuthProvider persist their values with this Hook.
// TODO (Lab 7.2 step 4): the Zustand persist middleware replaces it for tasks.
import { useEffect, useState } from "react";

export function useLocalStorage<T>(key: string, initialValue: T) {
  const [value, setValue] = useState<T>(() => {
    const stored = localStorage.getItem(key);
    return stored !== null ? (JSON.parse(stored) as T) : initialValue;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue] as const;
}
```

src/index.css

```
/* TODO (Lab 7.1 step 3): define colours as CSS variables in :root and override them in [data-theme="dark"]. */
/* Global styles: a simple starting point for TaskBoard */
body {
  margin: 0;
  font-family: system-ui, sans-serif;
  line-height: 1.5;
  color: #1f2937;
  background: #f9fafb;
}

nav a.active { font-weight: bold; }
```

src/main.tsx

```
// TODO (Lab 7.1 step 7): wrap <RouterProvider> in <AuthProvider> and <ThemeProvider>.
// TODO (Lab 7.3 step 3): create a QueryClient (staleTime 30_000) and add <QueryClientProvider> and <ReactQueryDevtools />.
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { createBrowserRouter } from "react-router";
import { RouterProvider } from "react-router-dom";
import "./index.css";
import Layout from "./pages/Layout";
import Dashboard from "./pages/Dashboard";
import Project from "./pages/Project";
import Settings from "./pages/Settings";
import NotFound from "./pages/NotFound";
import Login from "./pages/Login";
import RequireAuth from "./components/RequireAuth";

const router = createBrowserRouter([
  {
    path: "/",
    element: <Layout />,
    errorElement: <NotFound />,
    children: [
      { index: true, element: <Dashboard /> },
      { path: "projects/:projectId", element: <Project /> },
      { path: "settings", element: <RequireAuth><Settings /></RequireAuth> },
      { path: "login", element: <Login /> },
    ],
  },
]);

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <RouterProvider router={router} />
  </StrictMode>
);
```

src/pages/Dashboard.tsx

```
// TODO (Lab 7.2 steps 5-7): stop reading Outlet context; the board reads tasks from the store.
// The search can stay in the URL; add an assignee filter from a filter store.
import { useOutletContext, useSearchParams } from "react-router";
import ProjectBoard from "../components/ProjectBoard";
import { projects } from "../data/projects";
import type { BoardContext } from "../Layout";

export default function Dashboard() {
  const { tasks } = useOutletContext<BoardContext>();
  const [searchParams, setSearchParams] = useSearchParams();
  const q = searchParams.get("q") ?? "";

  const visible = tasks.filter((t) =>
    t.title.toLowerCase().includes(q.toLowerCase())
  );

  function handleSearch(value: string) {
    setSearchParams(value ? { q: value } : {});
  }

  return (
    <section>
      <h1>Dashboard</h1>
      <div className="search">
        <label htmlFor="search">Search tasks</label>
        <input id="search" type="search" value={q}
          onChange={(e) => handleSearch(e.target.value)} />
      </div>
      <p className="hint">New tasks added here go into the {projects[0].name} project.</p>
      <ProjectBoard tasks={visible} projectId={projects[0].id} />
    </section>
  );
}
```

src/pages/Layout.tsx

```
// TODO (Lab 7.1 step 3): add a theme toggle button to the nav.
// TODO (Lab 7.2 step 5): remove the tasks state, the seeding effect, BoardContext and the Outlet context.
import { useEffect, useState } from "react";
import type { Dispatch, SetStateAction } from "react";
import { Link, NavLink, Outlet } from "react-router";
import "../App.css";
import Header from "../components/Header";
import { useAuth } from "../hooks/useAuth";
import { useLocalStorage } from "../hooks/useLocalStorage";
import { projects } from "../data/projects";
import { SEED_URL, toTasks } from "../data/seed";
import type { Todo } from "../data/seed";
import type { Task } from "../types";

export type BoardContext = {
  tasks: Task[];
  setTasks: Dispatch<SetStateAction<Task[] | null>>;
};

export default function Layout() {
  const [tasks, setTasks] = useLocalStorage<Task[] | null>("tasks", null);
  const { user, logout } = useAuth();
  const [seedAttempt, setSeedAttempt] = useState(0);
  const [seedError, setSeedError] = useState<string | null>(null);
  const needsSeed = tasks === null;

  useEffect(() => {
    if (!needsSeed) return;
    const controller = new AbortController();
    fetch(SEED_URL, { signal: controller.signal })
      .then((res) => {
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        return res.json();
      })
      .then((todos: Todo[]) => setTasks(toTasks(todos)))
      .catch((cause: unknown) => {
        if (!controller.signal.aborted) {
          setSeedError(cause instanceof Error ? cause.message : String(cause));
        }
      });
    return () => controller.abort();
  }, [needsSeed, seedAttempt, setTasks]);

  return (
    <div className="app">
      <Header tasks={tasks ?? []} />
      <nav>
        <NavLink to="/" end>Dashboard</NavLink>
        {projects.map((p) => (
          <NavLink key={p.id} to={`/${p.id}`}>{p.name}</NavLink>
        ))}
        <NavLink to="/settings">Settings</NavLink>
        <span className="user">
          {user ? (
            <>
              Signed in as {user.name}{""}
              <button onClick={logout}>Log out</button>
            </>
          ) : (
            <Link to="/login">Log in</Link>
          )}
        </span>
      </nav>
    </div>
  );
}
```

```

    <main>
      {tasks === null ? (
        seedError ? (
          <p role="alert">
            Could not load starter tasks: {seedError}{" "}
            <button onClick={() => {
              setSeedError(null);
              setSeedAttempt((attempt) => attempt + 1);
            }}>Retry</button>
          </p>
        ) : (
          <p>Loading starter tasks...</p>
        )
      ) : (
        <Outlet context={{ tasks, setTasks } satisfies BoardContext} />
      )}
    </main>
  </div>
);
}

```

src/pages/Login.tsx

```

import { useState } from "react";
import type { SubmitEvent } from "react";
import { useLocation, useNavigate } from "react-router";
import { useAuth } from "../hooks/useAuth";

export default function Login() {
  const [name, setName] = useState("");
  const { login } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();
  const from = (location.state as { from?: string } | null)?.from ?? "/";

  function handleSubmit(e: SubmitEvent<HTMLFormElement>) {
    e.preventDefault();
    login(name.trim() || "Demo user");
    navigate(from, { replace: true });
  }

  return (
    <form className="login-form" onSubmit={handleSubmit}>
      <h1>Log in</h1>
      <label htmlFor="name">Name</label>
      <input
        id="name"
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Your name"
      />
      <button type="submit">Log in</button>
    </form>
  );
}

```

src/pages/NotFound.tsx

```
import { isRouteErrorResponse, Link, useRouteError } from "react-router";

export default function NotFound() {
  const error = useRouteError();
  const title = isRouteErrorResponse(error)
    ? `${error.status} ${error.statusText}`
    : "Something went wrong";

  return (
    <section>
      <h1>{title}</h1>
      <Link to="/">Back to dashboard</Link>
    </section>
  );
}
```

src/pages/Project.tsx

```
// TODO (Lab 7.2 step 6): read tasks with a store selector instead of Outlet context.
// TODO (Lab 7.3 step 5): read tasks with useQuery instead.
import { Link, useOutletContext, useParams } from "react-router";
import ProjectBoard from "../components/ProjectBoard";
import { projects } from "../data/projects";
import type { BoardContext } from "../Layout";

export default function Project() {
  const { projectId } = useParams();
  const { tasks } = useOutletContext<BoardContext>();
  const project = projects.find((p) => p.id === projectId);

  if (!project) {
    return (
      <section>
        <h1>Project not found</h1>
        <Link to="/">Back to dashboard</Link>
      </section>
    );
  }

  const projectTasks = tasks.filter((t) => t.projectId === projectId);
  return (
    <section>
      <h1>{project.name}</h1>
      <p>{projectTasks.length} tasks</p>
      <ProjectBoard tasks={projectTasks} projectId={project.id} />
    </section>
  );
}
```

src/pages/Settings.tsx

```
// TODO (Lab 7.2 step 5): Reset board uses Layout's Outlet context, which this lab removes.
import { useOutletContext } from "react-router";
import type { BoardContext } from "../Layout";

export default function Settings() {
  const { setTasks } = useOutletContext<BoardContext>();

  return (
    <section>
      <h1>Settings</h1>
      <p>Reset the board to load the starter tasks again.</p>
      <button className="reset-button" onClick={() => setTasks(null)}>Reset board</button>
    </section>
  );
}
```

src/types.ts

```
export type Status = "todo" | "doing" | "done";

export interface Task {
  id: string;
  title: string;
  status: Status;
  points: number;
  assignee?: string;      // optional
  tags?: string[];
  projectId: string;
}
```

tsconfig.app.json

```
{
  "compilerOptions": {
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.app.tsbuildinfo",
    "target": "es2023",
    "lib": ["ES2023", "DOM"],
    "module": "esnext",
    "types": ["vite/client"],
    "allowArbitraryExtensions": true,
    "skipLibCheck": true,

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": true,
    "moduleDetection": "force",
    "noEmit": true,
    "jsx": "react-jsx",

    /* Linting */
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src"]
}
```

tsconfig.json

```
{
  "files": [],
  "references": [
    { "path": "./tsconfig.app.json" },
    { "path": "./tsconfig.node.json" }
  ]
}
```

tsconfig.node.json

```
{
  "compilerOptions": {
    "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.node.tsbuildinfo",
    "target": "es2023",
    "lib": ["ES2023"],
    "types": ["node"],
    "skipLibCheck": true,

    /* Bundler mode */
    "module": "nodenext",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": true,
    "moduleDetection": "force",
    "noEmit": true,

    /* Linting */
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "erasableSyntaxOnly": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["vite.config.ts"]
}
```

vite.config.ts

```
import react from '@vitejs/plugin-react'
import { defineConfig } from 'vite'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})
```